

34. 在排序数组中查找元素的第一个和最后一个位置

通过 88 / 88 个通过的测试用例
白日梦想家 提交于 2026.03.12 21:46

执行用时分布

0 ms 击败 100.00%

消耗内存分布

17.16 MB 击败 77.52%



150%
100%
50%
0%

1ms 2ms 3ms 4ms

代码 C++

```
1 class Solution {
2     int findFirst(vector<int>& v, int target) {
3         int left = 0, right = v.size() - 1, first = -1;
4         while (left <= right) {
5             int mid = left + (right - left) / 2;
6             if (v[mid] == target) {
7                 first = mid;
8                 right = mid - 1;
9             } else if (v[mid] < target) {
10                left = mid + 1;
11            } else {
12                right = mid - 1;
13            }
14        }
15        return first;
16    }
17
18    int findLast(vector<int>& v, int target) {
19        int left = 0, right = v.size() - 1, last = -1;
20        while (left <= right) {
21            int mid = left + (right - left) / 2;
22            if (v[mid] == target) {
23                last = mid;
24                left = mid + 1;
25            } else if (v[mid] < target) {
26                left = mid + 1;
27            } else {
28                right = mid - 1;
29            }
30        }
31        return last;
32    }
33 }
34
```

更多挑战

278. 第一个错误的版本

2055. 蜡烛之间的盘子

2089. 找出数组排序后的目标下标

代码

```
1 class Solution {
2     int findFirst(vector<int>& v, int target) {
3         int left = 0, right = v.size() - 1, first = -1;
4         while (left <= right) {
5             int mid = left + (right - left) / 2;
6             if (v[mid] == target) {
7                 first = mid;
8                 right = mid - 1;
9             } else if (v[mid] < target) {
10                left = mid + 1;
11            } else {
12                right = mid - 1;
13            }
14        }
15        return first;
16    }
17
18    int findLast(vector<int>& v, int target) {
19        int left = 0, right = v.size() - 1, last = -1;
20        while (left <= right) {
21            int mid = left + (right - left) / 2;
22            if (v[mid] == target) {
23                last = mid;
24                left = mid + 1;
25            } else if (v[mid] < target) {
26                left = mid + 1;
27            } else {
28                right = mid - 1;
29            }
30        }
31        return last;
32    }
33 }
34
```

行 1, 列 1 已存储 运行 提交

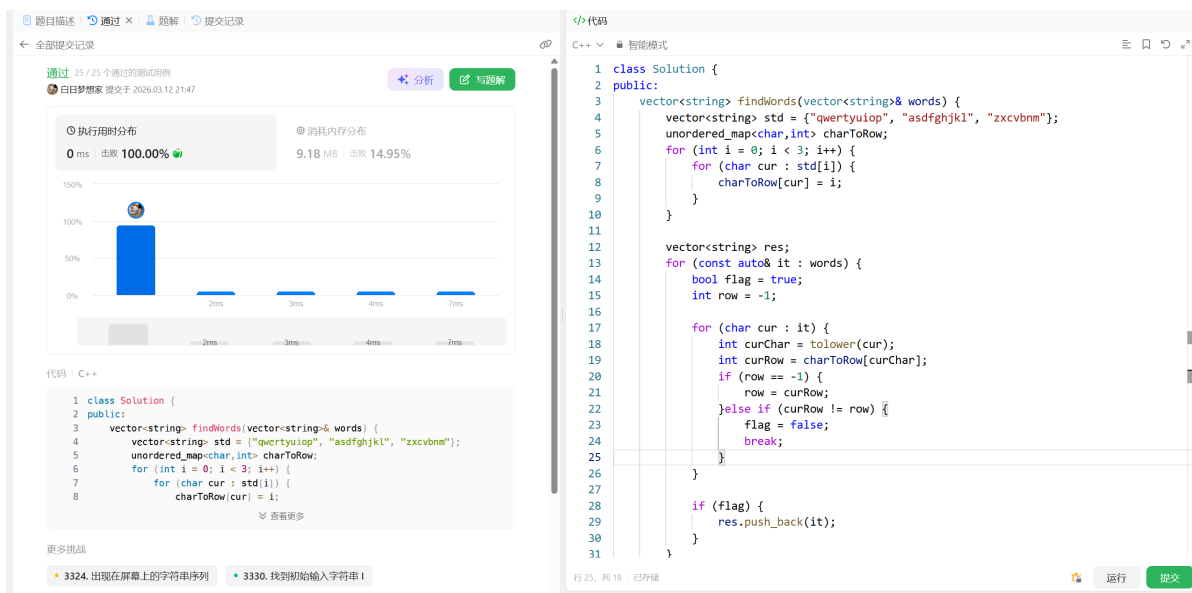
```
1 class Solution {
2     // 查找目标值 target 在数组中第一次出现的位置（左边界）
3     // 若未找到，返回 -1
4     int findFirst(vector<int>& v, int target) {
5         int left = 0, right = v.size() - 1, first = -1;
6         while (left <= right) {
7             int mid = left + (right - left) / 2;
8             if (v[mid] == target) {
9                 first = mid;           // 记录当前位置
10                right = mid - 1;       // 继续向左搜索更早的出现位置
11            } else if (v[mid] < target) {
12                left = mid + 1;
13            } else {
14                right = mid - 1;
15            }
16        }
17        return first;
18    }
19
20    // 查找目标值 target 在数组中最后一次出现的位置（右边界）
21    // 若未找到，返回 -1
22    int findLast(vector<int>& v, int target) {
23        int left = 0, right = v.size() - 1, last = -1;
24        while (left <= right) {
25            int mid = left + (right - left) / 2;
26            if (v[mid] == target) {
27                last = mid;           // 记录当前位置
28                left = mid + 1;       // 继续向右搜索更晚的出现位置
29            } else if (v[mid] < target) {
30                left = mid + 1;
31            } else {
32                right = mid - 1;
33            }
34        }
35    }
36 }
```

```

35         return last;
36     }
37
38 public:
39     // 主函数: 返回 target 在 nums 中的起始和结束位置
40     // 若 target 不存在, 返回 [-1, -1]
41     vector<int> searchRange(vector<int>& nums, int target) {
42         int first = findFirst(nums, target);
43         int last = findLast(nums, target);
44         return {first, last};
45     }
46 };

```

500. 键盘行



```

1 class Solution {
2 public:
3     vector<string> findWords(vector<string>& words) {
4         // 键盘三行字母（小写），用于构建字符到行号的映射
5         vector<string> std = {"qwertyuiop", "asdfghjkl", "zxcvbnm"};
6         unordered_map<char, int> charToRow;
7         // 遍历每一行，将每个字符映射到对应的行索引（0，1，2）
8         for (int i = 0; i < 3; i++) {
9             for (char cur : std[i]) {
10                 charToRow[cur] = i;
11             }
12        }
13
14        vector<string> res;
15        // 遍历输入的每个单词，判断是否所有字母都来自键盘的同一行
16        for (const auto& it : words) {
17            bool flag = true; // 标记当前单词是否满足条件
18            int row = -1; // 记录该单词第一个字符所在的行号
19
20            for (char cur : it) {
21                int curChar = tolower(cur); // 转为小写以匹配映射表
22                int curRow = charToRow[curChar]; // 获取当前字符所在行
23                if (row == -1) {

```

```
24         row = curRow;                // 第一个字符确定目标行
25     } else if (curRow != row) {
26         flag = false;                // 后续字符行号不同，不符合要求
27         break;
28     }
29 }
30
31 if (flag) {
32     res.push_back(it);                // 所有字符在同一行，加入结果
33 }
34 }
35
36 return res;
37 }
38 };
```